Name: Tedi Shaho

Professor: Janghoon Yang

CMPSC 463 - Project 1

March 15, 2026

Table of Contents

Overview …… 3

Dataset Description ……. 3

Installation and Usage Instructions …….3

Code Structure Explanation and Algorithm Descriptions …… 4

Execution Results …… 10

Discussion and Conclusion ……..13

Sources …… 16

Overview

The main objective of the project was to design and implement an algorithm-based system which deals with real-time sensor data of a water pump machine. The main algorithms which the system used to perform operations on the data were Divide and Conquer, Closest Pair and the Kadane (maximum subarray) algorithm. There were 52 sensors to take data from, with timestamps being 1 minute apart. In addition, the table had one final column which had information about the RUL, which stands for the Remaining Useful Life of each particular sensor, which basically indicated their overall health, and it gave insight as to how far the next potential maintenance would be. Sensors that scored high, had a high remaining life, and therefore had a low chance of having the need to be repaired or replaced. The RULs were split into different quantiles, Q10, Q40, and Q90, and sensors would be categorized based on which quantile they would fall on.

Dataset Description

The core implementation in Python was separated into 3 separate files, each having a specific task. Task 1 of the project had to do with dividing 10 randomly selected sensors, which computed the variance of a particular segment, and finally computed the Segmentation Complexity Score by counting the total number of resulting segments. The algorithm which was used for Task 1 was Divide and Conquer. Moving on, Task 2's main objective was to split all 10,000 time instances of data into 4 clusters, and to essentially compare those with the previously constructed 4 categories of RUL health data, and evaluate whether the clustering is a valid representation of the actual sensor machine health condition. Lastly, Task 3 had to do with implementing the Maximum Subarray algorithm, and in the process computed the absolute-first difference, and then subtracting the average of these absolute differences. Then, the Kadane algorithm would be implemented to find the maximum-sum subarray, which was the interval with the most deviation in sensor signals, which could be a good indicator of potential failure. Lastly, this section determines which category from the RUL appears the most, as this can be used to detect sensors which might need to be repaired.

Installation and Usage Instructions

1. Load Task1.py, and run it.
2. Load Task2.py, and run it.
3. Load Task3.py, and run it.

Code Structure Explanation & Algorithm Descriptions

The code is divided into 3 parts or tasks, where each task has a specific objective, correlating with the project instructions. This practice achieves modularity, as it is easier to break down the main problem into smaller subproblems. In addition, it makes the code easier to read, and cleaner.

Task 1:

```
1  import pandas as pd
2  import random
3  import numpy as np
4
5  # Path to my ZIPPED CSV file which has the sensor data and the timestamps from Kaggle
6  zip_file_path = r"C:\Users\Tedi\Downloads\rul_hrs.csv.zip"
7
8  # Part to read the CSV file directly from ZIP
9  df = pd.read_csv(zip_file_path, compression='zip', nrows=10000)
```

Lines 1-3 are the main library importations. The ‘random’ library is primarily used to randomly choose 10 of the sensors to perform the analysis on. The ‘pandas’ library is useful for loading and reading the data from the CSV file which contains the sensor readings. The ‘numpy’ library helps with the numerical calculations, like the mean and variance. Line 6 is the path of the CSV file from the directory of the computer. Then, line 9 deals with actually reading the file, where the ‘nrows’ = 10000’ parameter makes use of only the first 10,000 lines. The variable ‘df’ refers to the file containing the data of the sensors.

```
11  # 4 categories of the quantiles
12  Q10 = df['rul'].quantile(0.1)
13  Q50 = df['rul'].quantile(0.5)
14  Q90 = df['rul'].quantile(0.9)
    1 usage
15  def rul_category(rul):
16      if rul < Q10:
17          return "Extremely Low RUL"
18      elif rul < Q50:
19          return "Moderately Low RUL"
20      elif rul < Q90:
21          return "Moderately High RUL"
22      else:
23          return "Extremely High RUL"
24
25  df['rul_category'] = df['rul'].apply(rul_category)
```

Next, the Q10, Q50, and Q90 are the 10%, 50%, and 90% quantiles of the RUL from the data table. The function ‘rul_category” categorizes class labels, depending on where the value of the ‘rul’ falls, with simple if and elif statements. Line 25 is responsible for applying the function to each row, and creates a new column in the data set.

```
27  # Check category distribution
28  print(df['rul_category'].value_counts())
29  # Use random library to select 10 sensors
30  sensor_columns = [col for col in df.columns if "sensor" in col]
31  selected_sensors = random.sample(sensor_columns, k: 10)
32  print("Selected sensors for analysis:", selected_sensors)
33
```

Line 27 will print the amount of rows on the data table which correspond to each category. Then, the random library is used to select 10 out of the 52 sensors randomly, and these will be the ones which the analysis will be performed on.

```
36  def divide_and_conquer(signal, start=0, end=None, threshold=0.5):
37      if end is None:
38          end = len(signal)
39      segment = signal[start:end]
40      var = np.var(segment)
41      # If variance exceeds threshold, split further
42      if var > threshold and (end - start) > 1:
43          mid = start + (end - start) // 2
44          left_segments = divide_and_conquer(signal, start, mid, threshold)
45          right_segments = divide_and_conquer(signal, mid, end, threshold)
46          return left_segments + right_segments
47      else:
48          return [(start, end)]  # stable segment
```

This is the main part of Task 1, which is the implementation of the recursive segmentation algorithm. The main recursive logic is when the variance > threshold, which is when the split into halves begins. This is done with the if-statement from Line 42-46, where the ‘divide_and_conquer’ function is called recursively.

```
50  # Step 2: Segment the 10 sensors
51  segmentation_results = {}  # store segments per sensor
52  complexity_scores = {}  # store number of segments
53  # Threshold for splitting
54  threshold = 0.5
55  for sensor in selected_sensors:
56      signal = df[sensor].values
57      segments = divide_and_conquer(signal, threshold=threshold)
58      segmentation_results[sensor] = segments
59      complexity_scores[sensor] = len(segments)
60  # Step 3: Print the Segmentation Complexity score
61  print("Segmentation Complexity Scores (number of segments per sensor):")
62  for sensor, score in complexity_scores.items():
63      print(f"{sensor}: {score}")
```

This code deals with the segmentation of the 10 sensors, where the dictionary on line 51 stores the list of segments for each sensor. The ‘complexity_scores’ variable stores the number of segments. The for loop is used to get the sensor readings, apply the ‘divide_and_conquer’ function, and store and count the number of segments. Lastly, lines 60-63 prints the total number of segments for each sensor.

Task 2:

```python
def split_cluster(indices):
    cluster_points = sensor_data[indices]
    variances = np.var(cluster_points, axis=0)
    split_dim = np.argmax(variances)
    median_value = np.median(cluster_points[:, split_dim])

    left, right = [], []
    for idx in indices:
        if sensor_data[idx, split_dim] <= median_value:
            left.append(idx)
        else:
            right.append(idx)
    return left, right
```

This function computes the variance of each sensor and selects the one with maximum variance over the data set. The ‘indices’ parameter corresponds to the list of the row indices. After finding the median value, it will split points which are less than or equal than the median to the left, and points that are greater than or equal than the median to the right, and finally returns both of them.

```python
46 clusters = [list(range(len(sensor_data)))]
47 while len(clusters) < 4:
48     largest_cluster = max(clusters, key=len)
49     clusters.remove(largest_cluster)
50     left, right = split_cluster(largest_cluster)
51     clusters.append(left)
52     clusters.append(right)
53 print("Number of clusters:", len(clusters))
54 print("\nMajority RUL category per cluster:\n")
55 for i, cluster in enumerate(clusters):
56     cluster_categories = df.iloc[cluster]['rul_category']
57     counts = cluster_categories.value_counts()
58     majority_class = counts.idxmax()
59     majority_count = counts.max()
60     print(f"Cluster {i+1}")
61     print("Size:", len(cluster))
62     print("Majority class:", majority_class)
63     print("Majority count:", majority_count)
64     print("Class distribution:")
65     print(counts)
66     print()
```

This part of Task 2 first creates a list called ‘clusters’ which will have a list of 4 clusters inside of it, and Line 53 prints the number of clusters (4) at the end. Then, the for loop is responsible for getting the RUL categories for the points in the cluster (line 56), and the ‘counts’ variable reports

how many points belong to each category. Then, lines 58 and 59 are the most frequent RUL categories in the cluster and the number of points in the majority. Lastly, lines 60-66 print the cluster size, and the dominant RUL category.

Task 3:

```
def preprocess_signal(signal):
    d = np.abs(np.diff(signal))
    mean_d = np.mean(d)
    x = d - mean_d
    return x
```

This function computes the absolute-first difference, with the given formula of the project instructions. Then, it computes the average of the absolute differences, and finally subtracts the average of these absolute differences.

```
def kadane(arr):
    max_sum = arr[0]
    current_sum = arr[0]
    start = 0
    end = 0
    temp_start = 0
    for i in range(1, len(arr)):
        if current_sum + arr[i] < arr[i]:
            current_sum = arr[i]
            temp_start = i
        else:
            current_sum = current_sum + arr[i]
        if current_sum > max_sum:
            max_sum = current_sum
            start = temp_start
            end = i
    return max_sum, start, end
```

```
48  print("Kadane Results for All Sensors")
49  print("")
50  for sensor in sensor_columns:
51      signal = df[sensor].values
52      x = preprocess_signal(signal)
53      max_sum, start, end = kadane(x)
54      interval_categories = df.iloc[start:end+1]['rul_category']
55      dominant_category = interval_categories.value_counts().idxmax()
56      print(sensor)
57      print("Interval:", start, "-", end)
58      print("Max deviation:", round(max_sum,2))
59      print("Dominant RUL category:", dominant_category)
60      print()
```

This is the main implementation of the Kadane (Maximum subarray) algorithm. This will compute the interval with the most deviation. The ‘max_sum is the deviation so far, and ‘current_sum’ is the deviation of the current segment. Then, the for loop will go through each element, and if the addition will decrease the efficiency, it will start over. If not, the current segment will be extended. The if-statement updates the ‘max_sum’, ‘start’ and ‘end’ variables if the new segment’s deviation is larger than the current largest, and returns them. Lastly, the algorithm is performed for each sensor, extracting each of the time series, and finds the interval of strongest deviation. Line 54 gets the machine state of that particular deviation period and line 55 finds the most common category for that interval. The results are printed.

Execution Results

Task 1

```
rul_category
Moderately High RUL    4000
Moderately Low RUL     4000
Extremely High RUL     1000
Extremely Low RUL      1000
Name: count, dtype: int64
Selected sensors for analysis: ['sensor_45', 'sensor_25', 'sensor_22', 'sensor_42', 'sensor_20', 'sensor_29', 'sensor_47', 'sensor_13', 'sensor_19', 'sensor_02']
Segmentation Complexity Scores (number of segments per sensor):
```

```
sensor_45: 4345
sensor_25: 7703
sensor_22: 6646
sensor_42: 347
sensor_20: 3603
sensor_29: 9429
sensor_47: 1252
sensor_13: 696
sensor_19: 7216
sensor_02: 19
```

Task 2

```
Cluster 2
Size: 2500
Majority class: Moderately Low RUL
Majority count: 1631
Class distribution:
rul_category
Moderately Low RUL    1631
Extremely Low RUL      866
Extremely High RUL       3
```

```
Cluster 1
Size: 2500
Majority class: Moderately High RUL
Majority count: 1435
Class distribution:
rul_category
Moderately High RUL    1435
Extremely High RUL      997
Extremely Low RUL        60
Moderately Low RUL        8
```

```
Cluster 3
Size: 2500
Majority class: Moderately Low RUL
Majority count: 1392
Class distribution:
rul_category
Moderately Low RUL     1392
Moderately High RUL    1077
Extremely Low RUL        31
```

```
Cluster 4
Size: 2500
Majority class: Moderately High RUL
Majority count: 1488
Class distribution:
rul_category
Moderately High RUL    1488
Moderately Low RUL      969
Extremely Low RUL        43
```

```
Closest Pair Analysis Within Clusters:

Cluster 1: Closest pair of points = 0, 1, distance = 0.00
Cluster 2: Closest pair of points = 41, 42, distance = 0.00
Cluster 3: Closest pair of points = 76, 77, distance = 0.00
Cluster 4: Closest pair of points = 8, 9, distance = 0.00
```

Task 3

```
sensor_00
Interval: 62 - 4830
Max deviation: 9.49
Dominant RUL category: Moderately High RUL

sensor_01
Interval: 1 - 2038
Max deviation: 184.0
Dominant RUL category: Moderately High RUL
```

```
sensor_02
Interval: 8990 - 9825
Max deviation: 19.57
Dominant RUL category: Extremely Low RUL

sensor_03
Interval: 5217 - 9488
Max deviation: 32.34
Dominant RUL category: Moderately Low RUL
```

```
sensor_04
Interval: 736 - 5914
Max deviation: 1042.73
Dominant RUL category: Moderately High RUL

sensor_05
Interval: 102 - 4789
Max deviation: 859.84
Dominant RUL category: Moderately High RUL
```

```
sensor_04
Interval: 736 - 5914
Max deviation: 1042.73
Dominant RUL category: Moderately High RUL

sensor_05
Interval: 102 - 4789
Max deviation: 859.84
Dominant RUL category: Moderately High RUL
```

```
sensor_51
Interval: 4577 - 7868
Max deviation: 630.02
Dominant RUL category: Moderately Low RUL
```

Discussion & Conclusion

Task 1, Step 5: I would say the segmentation results relate to the RUL categories, as they can be good indicators for how frequently the sensor changes its signal. For instance, the sensors which have higher segment score complexity can be associated more often with the times where the sensor scores on the Low rul values.

Task 1, Step 6: The variability across different sensors makes it difficult to generalize exactly how each sensor will behave over time, as there are both stable and unstable periods. There are sensors which have more segments than others, and this indicates more unstable behavior. There are other sensors which have less variability in behavior. Perhaps if the sample of rows was smaller, there could be a more precise way to generalize the behavior of the sensors. Maybe even if 25 sensors were chosen instead of 52, there could be less change in behavior across all of the sensors.

Task 2, Step 3: I will use the sample execution results provided in the section above. For Cluster 1, the dominating class is "Moderately High", indicating overall proper functioning sensors, and indicates low possibility of defect. For Cluster 2, the majority class was Moderately Low, which indicates that there might be a higher possibility of more sensors with defects. For Cluster 3, the majority class was Moderately Low, which again signals aging sensors which might need replacement soon. Lastly, for Cluster 4, the majority class was Moderately High, which for the most part, should be good.

Task 3, Step 5: I would say the sensors which provide good early indicators of low URL are the ones that fall into the Moderately Low/Extremely Low RUL categories, as they have the sensors which might have the most defects. For instance, according to the sample execution results, sensors 2, 3, and 51 are good early indicators of low RUL, as they fall into the category.

One challenge in the completion of this project was the implementation of the Kadane Algorithm. I believe this was the most challenging because it was the algorithm which I was the least familiar with, and I had to spend some time to fully understand and familiarize myself with it. Not only that, but I think the for-loop with the if-statements took me some time to implement the logic correctly. For instance, in the beginning, I would set the for loop to begin from 0 all the way to the length of the array, instead of starting from 1. Another thing I would do wrong is forget to include the else statement to update the value of the 'current_sum' variable. Another challenge in the project was loading the dataset and reading it. For some reason, I was really struggling in the beginning and I kept getting errors, so I just wrote down the path to the ZIP file, and it worked. One possible improvement to the project is adding a Sorting section where the sensor's health would be quantified in some way (say from 1-100), with 1 being bad, and 100

being excellent. Then, the sensors would be sorted from the ones in poorest condition to ones in the best condition.

Sources

1. "Find the Closest Pair from Two Sorted Arrays." *GeeksforGeeks*, GeeksforGeeks, 26 Feb. 2026, www.geeksforgeeks.org/dsa/given-two-sorted-arrays-number-x-find-pair-whose-sum-closest-x/.
2. "Maximum Subarray Sum - Kadane's Algorithm." *GeeksforGeeks*, GeeksforGeeks, 22 July 2025, www.geeksforgeeks.org/dsa/largest-sum-contiguous-subarray/.